



جامعة الأميرة نورة بنت عبد الرحمن
Princess Nourah bint Abdulrahman University

Search title:

Cryptography by ring

Supervised By: professor Nazek Alessa

Submitted By:

Mashael alnafea 441002460

Raghad Alghamdi 441002275

Rawan aloraini 442007397

Alanood Sahara 439005328

Dana Ali Al-hefdhi 442001372

Acknowledgment and Appreciation

I extend my sincere gratitude and appreciation to everyone who contributed to the completion of this project, starting with my professors who provided me with knowledge and guidance, my family who supported me with love and patience, and my colleagues who shared this journey and enriched it. This work is the result of all your efforts, and for that, you have my deepest gratitude and appreciation.

Abstract

With the growing need for secure communication, cryptography plays a vital role in protecting digital information. Two prominent public-key cryptosystems are RSA and NTRU. RSA, developed in 1978, is based on the difficulty of factoring large prime numbers, offering strong security but requiring large key sizes for protection against modern attacks. In contrast, NTRU, introduced in 1998, is a lattice-based cryptosystem known for its speed and efficiency, especially in constrained environments. NTRU is also considered more resilient to quantum attacks due to the hardness of lattice problems. This work compares RSA and NTRU in terms of structure, performance, and security in both classical and post-quantum contexts.

Contents

Abstract	ii
Introduction	1
Chapter 1	2
Mathematical Foundations of Cryptographic Fields	2
Mathematical Structures: Groups, Rings, and Fields	2
1.2.1 Definition	4
1.2.7 Definition	4
Chapter 2	9
RSA in Rings and Fields	9
Naming of RSA.....	9
Step 1: Convert (GOLDEN) to Numbers Using the Encoding Table	11
Step 2: Choose RSA Parameters We choose two large prime numbers p and q for RSA encryption:	11
Step 4: Convert the Ciphertext to Characters.....	13
2.2.1 Factorization Attack	13
Chapter 3	16
NTRU in Fields and Rings	16
Origin of the Name NTRU.....	16
Comparison Table: RSA vs. NTRU	19
Step 1: Convert "HELLO" to Numbers Using the Encoding Table	21
Step 2: Choose RSA Parameters	21
Step 3: Encrypt the Message Using the Public Key....	21
Step 4: Convert the Ciphertext to Characters	21
References.....	28

Introduction:

Cryptography has evolved over thousands of years, from simple ciphers used by ancient civilizations to the advanced systems securing today's digital communication. Classical methods like the Caesar and Vigenère ciphers laid the groundwork for modern encryption. A major breakthrough came in 1978 with the introduction of RSA by Rivest, Shamir, and Adleman, marking the birth of public-key cryptography.

In recent years, lattice-based systems such as NTRU have gained attention as strong candidates for future security, especially in the face of quantum computing threats. This study compares RSA and NTRU, focusing on their strengths, weaknesses, and potential as future-proof cryptographic systems.

CHAPTER

1

Mathematical Foundations of Cryptographic Fields

Encryption in fields refers to using mathematical fields, like Galois Fields, to secure data. A field is a set of numbers where you can perform operations such as addition, subtraction, multiplication, and division, and always get results that stay within that set. This is useful for encryption because it allows complex operations to be done with large numbers while keeping things simple and efficient. In cryptography, these fields help in creating strong security systems by allowing operations like multiplication and inversion to be performed in a way that is hard to reverse without the right key.

Mathematical Structures: Groups, Rings, and Fields

1.1 Group: A group is a set of elements combined with an operation (like addition or multiplication) that follows four main rules: closure, associativity, identity, and inverse .

1.1.1 Definition:

- Closure: If $a, b \in G$, then $ab \in G$.
- Associativity: $(ab)c = a(bc)$ for all $a, b, c \in G$.
- Identity Element: There exists an element e such that $ea = ae = a$ for all $a \in G$.
- Inverse Element: For each $a \in G$, there exists a^{-1} such that $aa^{-1} = a^{-1}a = e$.

1.1.2 Example [1]: The set of integers $\mathbb{Z}$ under addition forms a group.

1.1.3 Example [1]: The set of invertible 2×2 matrices under matrix multiplication.

1.1.4 Example [1]: The set of nonzero rational numbers under multiplication.

1.1.5 Subgroup: A subgroup is a smaller set within a group that also follows the same rules for the operation.

1.1.6 Definition: Subgroup Test:

A nonempty subset H of G is a subgroup if:

- Closure under operation: If $a, b \in H$, then $ab \in H$.
- Closure under inverses: If $a \in H$, then $a^{-1} \in H$.

1.1.7 Example [1]: The set of even integers $2\mathbb{Z}$ as a subgroup of $\mathbb{Z}$.

1.1.8 Example [1]: The set of upper triangular matrices inside the set of all invertible matrices.

1.1.9 Example [1]: The set of positive rational numbers under multiplication as a subgroup.

1.1.10 Corollary: The intersection of subgroups is a subgroup

1.2 Ring [3]: A ring is a set with two operations: addition and multiplication. The addition forms an Abelian group (meaning it's commutative), but multiplication doesn't always require an inverse for every element.

1.2.1 Definition

A ring R is a set with:

- R is an Abelian group under addition.
- R is closed and associative under multiplication.
- Distributive laws hold: $a(b+c) = ab + ac$ and $(a+b)c = ac + bc$.

1.2.2 Example[1]: The set of integers $\mathbb{Z}$ under addition and multiplication.

1.2.3 Example[1]: The set of all 2×2 matrices under matrix addition and multiplication.

1.2.4 Example[1]: The set of polynomials with integer coefficients under addition and multiplication.

1.2.5 Corollary : Every field is a commutative ring with unity.

1.2.6 Subring [3]: A subring is a smaller set within a ring that also follows the same rules for both operations .

1.2.7 Definition [3]:

Subring Test: A nonempty subset S of a ring R is a subring if:

- It is closed under subtraction and multiplication.
- It is associative and distributive over addition.

1.2.8 Example [1]: The set of even integers as a subring of the integers $\mathbb{Z}$.

1.2.9 Example [1]: The set of upper triangular matrices inside the set of all matrices .

1.2.10 Example [1]: The set of polynomials divisible by x as a subring of the polynomial ring.

1.2.11 Corollary : The intersection of subrings is a subring.

1.3 Field [4]: A field is a set where both addition and multiplication are defined, and every nonzero element has a multiplicative inverse .

1.3.1 Definition: A field F is a set with addition and multiplication where:

- F forms an Abelian group under addition.
- The nonzero elements form an Abelian group under multiplication.
- Distributive laws hold.

1.3.2 Example [1]: The set of rational numbers $\mathbb{Q}$ under addition and multiplication.

1.3.3 Example [1]: The set of real numbers $\mathbb{R}$ as an example of a field.

1.3.4 Example [1]: The set of complex number $\mathbb{C}$ s as a field.

1.3.5 Definition [2]: Integral Domain An integral domain is a commutative ring with unity and no zero-divisors

1.3.6 Corollary : Every finite integral domain is a field.

1.3.7 Subfield [3]: A subfield is a smaller set within a field that satisfies the same rules as the field itself .

1.3.8 Definition : A nonempty subset F of a field E is a subfield if:

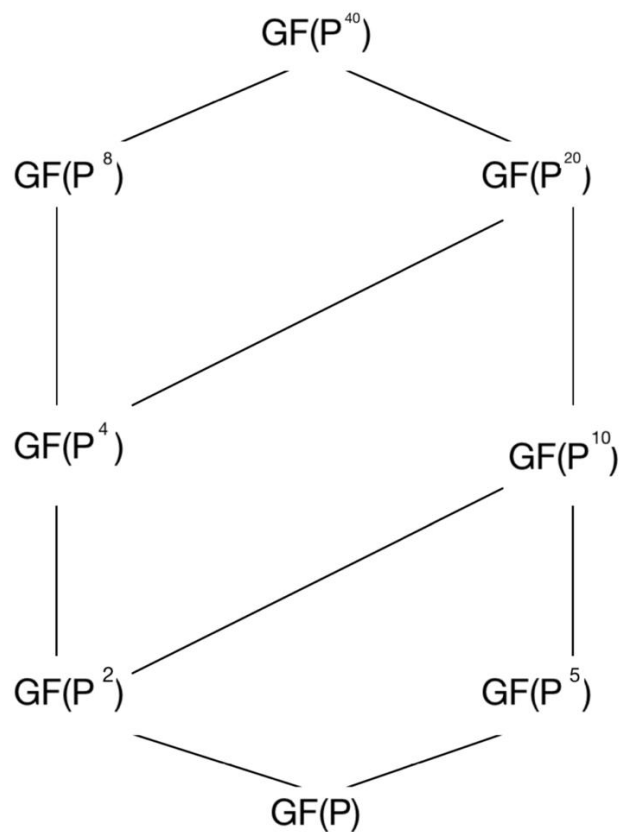
- F is closed under field operations.
- Every nonzero element of F has a multiplicative inverse in F .

1.3.9 Example[1]: The set of rational numbers $\mathbb{Q}$ as a subfield of real numbers $\mathbb{R}$.

1.3.10 Example[1]: The set of real numbers $\mathbb{R}$ as a subfield of complex numbers $\mathbb{C}$.

1.4 Galois Field [4]: A Galois Field (or finite field) is a field that contains a finite number of elements. Finite fields are fundamental in areas like cryptography because they allow operations to be performed with a limited set of elements, making calculations manageable. A Galois field is often denoted as $GF(p^n)$, where (p) is a prime number and (n) is a positive integer .

1.4.1 Example : Find The subfield lattice of **Galois Field** order $p^{(40)}(GF(p^{(40)}))$



1.5 polynomial ring [2]: is a mathematical structure formed by polynomials over a given ring, such as integers or real numbers. It is denoted by $R[x]$, where R is a ring, and x is the indeterminate (variable) used in the polynomial. The elements of the polynomial ring $R[x]$ are polynomials with coefficients from R and powers of the indeterminate x .

1.5.1 Definition [2]: Let R be a commutative ring. The set of formal symbols

$$R[x] = \{ a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \mid a_i \in R, n \text{ is a nonnegative integer} \}$$

is called the ring of polynomials over R in the indeterminate x .

Two elements

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

and

$$b_m x^m + b_{m-1} x^{m-1} + \cdots + b_1 x + b_0$$

of $R[x]$ are considered equal if and only if $a_i = b_i$ for all nonnegative integers i .

(Define $a_i = 0$ when $i > n$ and $b_i = 0$ when $i > m$.)

1.5.2 Examples [2]: To find the quotient and remainder upon dividing

$f(x) = 3x^4 + x^3 + 2x^2 + 1$ by $g(x) = x^2 + 4x + 2$, where $f(x)$ and $g(x)$ belong to $Z_5[x]$, we may proceed by long division, provided we keep in mind that addition and multiplication are done modulo 5. Thus

$3x^2 + 4x$ is the quotient and $2x + 1$ is the remainder. Therefore,

$$3x^4 + x^3 + 2x^2 + 1 = (x^2 + 4x + 2)(3x^2 + 4x) + 2x + 1.$$

1.5.3 Corollary [2]: Let F be a field, $a \in F$, and $f(x) \in F[x]$. Then $f(a)$ is the remainder in the division of $f(x)$ by $x - a$.

1.5.4 Corollary : Let F be a field, $a \in F$, and $f(x) \in F[x]$. Then a is a zero of $f(x)$ if and only if $x - a$ is a factor of $f(x)$.

CHAPTER

2

RSA in Rings and Fields

The RSA algorithm is one of the most widely used asymmetric encryption systems, relying on number theory and algebraic structures such as rings and fields. It is implemented within abstract algebra, utilizing integer rings and polynomial rings for encryption and decryption operations.

Naming of RSA

The RSA algorithm is named after its inventors:

- Ronald Rivest
- Adi Shamir
- Leonard Adleman

It was developed in 1977 at MIT as the first practically applicable public-key cryptographic system.

2.1 Euler Theorem[4]:

For any positive integer n , $\varphi(n)$ is defined as the number of integers k in the range $1 \leq k \leq n$ such that $\gcd(k, n) = 1$ where $\gcd$ is the greatest common divisor.

2.1.1 Example:

For $n = 9$, the integers less than or equal to 9 that are coprime with 9 are 1, 2, 4, 5, 7, 8. Therefore, $\varphi(9) = 6$.

2.1.2 Theorem Supporting RSA [4]

If n is a positive integer and a is coprime with n , then:

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

Where $\varphi(n)$ is Euler's Totient Function, representing the count of integers less than n that are coprime with it.

This theorem is used to compute the private key d such that:

$$de \equiv 1 \pmod{\varphi(n)}$$

This ensures that decryption using the private key restores the original message.

2.1.3 RSA Encryption and Decryption Functions [4]

RSA relies on modular exponentiation in ring structures:

Encryption function: $c = M^e \bmod n$ where:

- c is the ciphertext.
- M is the plaintext.
- e is the public key.
- $n = (p)(q)$ (product of two prime numbers).
- d is the private key

d is the Decryption function: $M = C^d \bmod n$

2.1.4 Example:

Encryption (GOLDEN) using RSA SYSTEM

We will use the following table to transfer the letters to numbers:

residues modulo 26 as follows: $A \leftrightarrow 0, B \leftrightarrow 1, \dots, Z \leftrightarrow 25$. Since we will be using this correspondence in several examples, let's record it for future use:

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12

N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

Step 1: Convert (GOLDEN) to Numbers Using the Encoding Table

We start by converting the word (GOLDEN) into numbers using the encoding table

where $A = 0, B = 1, \dots, Z = 25$. The conversion is as follows:

$$G \rightarrow 6, O \rightarrow 14, L \rightarrow 11, D \rightarrow 3, E \rightarrow 4, N \rightarrow 13$$

Thus, the message (GOLDEN) is represented as:

$$(6, 14, 11, 3, 4, 13)$$

Step 2: Choose RSA Parameters We choose two large prime numbers p and q for RSA encryption:

$$p = 11, q = 17$$

Calculate n and $\phi(n)$:

$$n = (p)(q) = (11)(17) = 187$$

$$\phi(n) = (p - 1) (q - 1) = (11 - 1) (17 - 1) = (10) (16) = 160$$

Next, choose $e = 7$ such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.

Now, we calculate the private key d such that:

$$e \cdot d \equiv 1 \text{ mod } \phi(n)$$

Using the extended Euclidean algorithm, we find:

$$d = 23$$

Thus, the public key is $(e, n) = (7, 187)$ and the private key is $d = 23$.

Step 3: Encrypt the Message Using the Public Key To encrypt the numbers, we use the RSA encryption formula:

$$C = M^e \text{ mod } n$$

where M is each number representing a character in the message, and C is the ciphertext.

$$C_1 = 6^7 \text{ mod } 187 = 1$$

$$C_2 = 14^7 \text{ mod } 187 = 27$$

$$C_3 = 11^7 \text{ mod } 187 = 61$$

$$C_4 = 3^7 \text{ mod } 187 = 67$$

$$C_5 = 4^7 \text{ mod } 187 = 58$$

$$C_6 = 13^7 \text{ mod } 187 = 19$$

Thus, the ciphertext is:

$$C = (1, 27, 61, 67, 58, 19)$$

Step 4: Convert the Ciphertext to Characters

Now, we take each number in the ciphertext and apply mod 26 to convert them to letters, using the encoding table where

$$A = 0, B = 1, \dots, Z = 25.$$

$$C_1 = 1 \bmod 26 = 1 \Rightarrow B$$

$$C_2 = 27 \bmod 26 = 1 \Rightarrow B$$

$$C_3 = 61 \bmod 26 = 9 \Rightarrow J$$

$$C_4 = 67 \bmod 26 = 15 \Rightarrow P$$

$$C_5 = 58 \bmod 26 = 6 \Rightarrow G$$

$$C_6 = 19 \bmod 26 = 19 \Rightarrow T$$

Thus, the encrypted message in letters is:

Encrypted message = BBJPJT

2.2 Weaknesses and Attacks on RSA [4]

Despite its strong mathematical foundation, RSA is vulnerable to several attacks if not properly implemented. Some of the key attacks include:

2.2.1 Factorization Attack [4]

The security of RSA relies on the difficulty of factoring large numbers. If an attacker can efficiently factor the modulus n (where $n = pq$), they can determine the private key d . Advanced factorization techniques such as the General Number Field Sieve (GNFS) pose a significant risk.

2.2.2 Timing Attack [10]

This attack exploits the time variations in RSA decryption operations. By analyzing the time taken to perform modular exponentiation, an attacker may infer bits of the private key.

Implementing countermeasures such as constant-time algorithms can mitigate this risk.

2.2.3 Low Exponent Attack [4]

If the public exponent e is too small (such as $e = 3$), it increases the risk of an attacker recovering the plaintext message using mathematical shortcuts. This can be mitigated by choosing a sufficiently large exponent, such as $e = 65537$.

2.3 Optimizations and Variants of RSA [4]

Several optimizations and modifications have been introduced to enhance the security and efficiency of RSA encryption.

2.3.1 RSA with Padding (OAEP) [4]

Optimal Asymmetric Encryption Padding (OAEP) is a padding scheme designed to prevent certain attacks, such as chosen ciphertext attacks. It introduces randomness into the encryption process, making it harder to decrypt messages without the private key.

2.3.2 Hybrid RSA Encryption [4]

Hybrid encryption combines RSA with symmetric encryption methods like AES. In this approach, RSA is used to encrypt a symmetric session key, which is then used for encrypting the actual message. This significantly improves performance while maintaining security.

2.3.3 Multi-Prime RSA [4]

Multi-prime RSA is an extension that uses more than two prime numbers (e.g., $n = p * q * r$).

This allows faster decryption and signing operations while maintaining the same level of security.

2.3.4 Corollaries [2]

- 1- RSA relies on ring structures for encryption, where computations are performed in $\mathbb{Z}/n\mathbb{Z}$.
- 2- Euler's theorem ensures correct encryption and decryption.
- 3- Choosing large prime numbers makes factorization attacks difficult.

CHAPTER

3

NTRU in Fields and Rings

NTRU is a lattice-based cryptographic system designed as a more efficient alternative to traditional encryption methods like RSA. It operates within polynomial rings, where corecipients belong to finite fields, allowing for high-speed encryption with lower resource consumption.

Origin of the Name NTRU

The name NTRU stands for Number Theoretic Ring Units, referring to the use of ring structures in its encryption scheme. NTRU relies on polynomial rings over modular arithmetic, making it resilient to quantum attacks.

3.1 Theorem Supporting NTRU [6]

NTRU is based on polynomial ring structures, formulated as follows: Theorem: If $R = \mathbb{Z}[x] / (x^n - 1)$, then R forms a ring containing number theoretic units suitable for encryption. This enables secure key generation and decryption mechanisms resistant to lattice-based attacks.

3.1.2 NTRU Encryption and Decryption Functions [6]

Step 1: Define Parameters

- p : Small modulus (prime number).
- q : Large modulus (prime number greater than p).
- N : Degree of the polynomial.
- $f(x)$: Private key polynomial (random polynomial).
- $g(x)$: Random polynomial.
- $m(x)$: Message polynomial (represents the message to be encrypted).

Step 2: Generate the Public Key $h(x)$

1. Compute the inverse of $f(x)$ modulo p :

$$f^{-1} \bmod p$$

2. Compute the public key $h(x)$ as:

$$h(x) = \frac{f^{-1}(x)g(x)}{p} \bmod q$$

where $g(x)$ is the random polynomial and $f(x)$ is the private key.

Step 3: Choose a Random Polynomial $r(x)$

Choose the random polynomial $r(x)$:

$$r(x) = x^2 + 2$$

Step 4: Encrypt the Message $m(x)$

To encrypt the message, we use the following formula:

$$e(x) = p \cdot r(x) \cdot h(x) + m(x) \mod q$$

where:

- $m(x)$ is the message polynomial.
- $r(x)$ is the random polynomial.
- $h(x)$ is the public key.

Step 5: Final Encrypted Message

The final encrypted message is:

$$e(x) = p \cdot r(x) \cdot h(x) + m(x) \mod q$$

3.1.3 Definition [6]

NTRU is an asymmetric encryption system based on polynomial ring operations, leveraging lattice-based cryptography for security against quantum attacks.

3.1.4 Corollaries : [6]

1-Every Element in the Ring Can Be Inverted

If $f(x)$ is an element of the ring, then its inverse exists such that: $f(x) f^{-1}(x) \equiv 1 \mod q$

2- NTRU is Resistant to Quantum Attacks

Since NTRU relies on lattice-based problems, it is inherently secure against quantum algorithms .

3- NTRU Reduces Key Size and Improves Performance

Compared to RSA, NTRU achieves faster encryption and decryption with significantly smaller keys, making it ideal for constrained environments.

Comprehensive Comparison Between RSA and NTRU

RSA and NTRU are two cryptographic systems with distinct mathematical foundations and practical applications. RSA, one of the earliest public-key cryptosystems, relies on the difficulty of integer factorization, whereas NTRU, a more modern alternative, is based on lattice-based cryptography, offering enhanced resistance against quantum attacks. This chapter provides a comprehensive comparison of these two encryption systems, highlighting their key differences in terms of security, performance, storage requirements, and real-world usability.

Comparison Table: RSA vs. NTRU

Metric	RSA	NTRU
Security	Vulnerable to quantum computing Attacks	Quantum-resistant due to lattice-based cryptography
Key Size	Large keys (2048-bit or higher recommended)	Smaller keys, reducing memory usage
Encryption/Decryption Speed	Slower due to large key sizes and complex computations	Faster due to polynomial operations
Mathematical Basis	Depends on integer factorization	Based on the hardness of lattice problems
Storage Requirement	Requires more storage due to large key sizes	More efficient storage with smaller keys

Computational Complexity	Exponential time complexity (difficult but breakable by quantum computers)	Lattice problems are NP-hard, making them infeasible for quantum attacks
Scalability	Requires increasing key sizes for stronger security, reducing efficiency	Can maintain efficiency with smaller keys
Suitability for IoT & Lightweight Devices	Less suitable due to computational Demands	Ideal for low-power devices due to efficiency
Resistance to Side Channel Attacks	Prone to certain types of side-channel attacks	More resilient to side channel attacks due to randomized operations
Post-Quantum Readiness	Not secure against quantum computers	Designed to resist quantum-based attacks

Note:-

- NTRU provides an alternative based on polynomial rings, offering better resistance to quantum attacks.
- RSA requires larger storage compared to NTRU but remains widely used due to its simplicity.

Illustrative Example:

Encrypting and Decrypting a Message Example:

RSA Encryption for the Word "HELLO"

Step 1: Convert "HELLO" to Numbers Using the Encoding Table

We start by converting the word "HELLO" into numbers using the encoding table where $A = 0, B = 1, \dots, Z = 25$. The conversion is as follows:

$$H \rightarrow 7, E \rightarrow 4, L \rightarrow 11, L \rightarrow 11, O \rightarrow 14$$

Thus, the message "HELLO" is represented as:

$$(7, 4, 11, 11, 14)$$

Step 2: Choose RSA Parameters

We choose two large prime numbers p and q for RSA encryption:

$$p = 11, q = 17$$

Calculate n and $\phi(n)$:

$$n = p \times q = 11 \times 17 = 187$$

$$\phi(n) = (p - 1) \times (q - 1) = (11 - 1) \times (17 - 1) = 10 \times 16 = 160$$

Next, choose $e = 7$ such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.

Now, we calculate the private key d such that:

$$e \cdot d \equiv 1 \pmod{\phi(n)}$$

Using the extended Euclidean algorithm, we find:

$$d = 23$$

Thus, the public key is $(e, n) = (7, 187)$ and the private key is $d = 23$

Step 3: Encrypt the Message Using the Public Key

To encrypt the numbers, we use the RSA encryption formula:

$$C = M^e \bmod n$$

where M is each number representing a character in the message, and C is the ciphertext.

$$C_1 = 7^7 \bmod 187 = 823543 \bmod 187 = 57$$

$$C_2 = 4^7 \bmod 187 = 16384 \bmod 187 = 58$$

$$C_3 = 11^7 \bmod 187 = 19487171 \bmod 187 = 61$$

$$C_4 = 11^7 \bmod 187 = 61$$

$$C_5 = 14^7 \bmod 187 = 105413504 \bmod 187 = 27$$

Thus, the ciphertext is:

$$C = (57, 58, 61, 61, 27)$$

Step 4: Convert the Ciphertext to Characters

Now, we take each number in the ciphertext and apply mod 26 to convert them to letters, using the encoding table where

$A = 0, B = 1, \dots, Z = 25$.

$$C_1 = 57 \bmod 26 = 5 \Rightarrow F$$

$$C_2 = 58 \bmod 26 = 6 \Rightarrow G$$

$$C_3 = 61 \bmod 26 = 9 \Rightarrow J$$

$$C_4 = 61 \bmod 26 = 9 \Rightarrow J$$

$$C_5 = 27 \bmod 26 = 1 \Rightarrow B$$

Thus, the encrypted message in letters is:

Encrypted message = FGJJB

NTRU Encryption for the Word "HELLO"

First, the parameters of the NTRU must be determined which are (N, p, q) , such that:

- N must be a prime number
- p and q are relatively prime, i.e. $\gcd(p, q) = 1$
- q be sufficiently larger than p

Next two polynomials must be found which are:

- $f(x)$ which must have an inverse, f_p , in mod p , and f_q , in mod q such that

$$ff_p = 1 \pmod{p} \text{ and } ff_q = 1 \pmod{q} \text{ in } \mathbb{Z}[x]/(x^N - 1)$$

- $g(x)$

Both polynomials' degrees must be at most $N-1$, with coefficients from the set $\{-1, 0, 1\}$

Finally, h is calculated as $h = pf_qg \pmod{q}$

- The public key is $\{h\}$
- The private key is $\{f, f_p\}$

While the parameters (N, P, Q) are made public.

- **Encryption of message m :** $e = rh + m \pmod{q}$, where r is any chosen polynomial used to obscure the message.
- **Decryption of e :** $m = f_p * a \pmod{p}$ centered where

$$a = f * e \pmod{q} \text{ centered}$$

Centering about p is ensuring that coefficients are in the range of $\left[-\frac{p}{2}, \frac{p}{2}\right]$

$$c_{\text{balanced}} = \begin{cases} c \bmod p & \text{if } (c \bmod p) \leq \left\lfloor \frac{p}{2} \right\rfloor, \\ (c \bmod p) - p & \text{if } (c \bmod p) > \left\lfloor \frac{p}{2} \right\rfloor. \end{cases}$$

Example Encryption and decryption of “Hello”

Setting the system

Choosing $(N, p, q) = (11, 3, 61)$

$$f = 1 + x + x^2 + x^4 - x^6 - x^8 - x^{10}$$

$$f_p = x + 2x^2 + 2x^3 + 2x^4 + x^5 + x^7 + x^9$$

$$f_q = 31 + 32x + 60x^2 + 24x^3 + 21x^4 + 47x^5 + 53x^6 + 40x^7 + 26x^8 + 49x^9 + 45x^{10}$$

$$g = 1 + x^2 + x^4 - x^6 - x^8 - x^9$$

$$\text{choosing } r = 1 + x + x^2 + x^7$$

Now computing $h = pf_qg \pmod{q}$

$$h = 28 + 15x^1 + 35x^2 + 47x^3 + 32x^4 + 37x^5 + 23x^6 + 16x^7 + 14x^8 + 25x^9 + 33x^{10}$$

Encryption of “Hello”

To encrypt the message, each letter has to be first transformed into a polynomial. This can be done based on the binary representation of the position of the letter in the alphabet. The binary values will be the coefficient of the polynomial

Letter	decimal	binary	Polynomial
H	7	111	$1 + x + x^2$
E	4	100	x^2
L	11	1011	$1 + x + x^3$
O	14	1110	$x + x^2 + x^3$

Now encrypt each letter.

Letter	$e = rh+m \pmod{61}$
H	$58 + 53x^1 + 41x^2 + 52x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$
E	$57 + 52x^1 + 41x^2 + 52x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$
L	$58 + 53x^1 + 40x^2 + 53x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$
L	$58 + 53x^1 + 40x^2 + 53x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$
O	$57 + 53x + 41x^2 + 53x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$

Decryption of the above encryption results (e_H, e_E, e_L, e_O)

e	$a = f * e \pmod{q}$ centered	$b = f_p a \pmod{p}$ centered	M
e_H	$3 + 4x + 6x^2 + 5x^3 + 5x^4 + x^5 - x^7 - 8x^8 - 4x^9 - 8x^{10}$	$1 + x + x^2$	H
e_E	$3 + 2x + 4x^2 + 4x^3 + 4x^4 + x^6 - 7x^8 - 3x^9 - 7x^{10}$	x^2	E
e_L	$2 + 5x + 4x^2 + 5x^3 + 5x^4 + 2x^5 - x^6 - 7x^8 - 5x^9 - 7x^{10}$	$1 + x + x^3$	L
e_L	$2 + 5x + 4x^2 + 5x^3 + 5x^4 + 2x^5 - x^6 - 7x^8 - 5x^9 - 7x^{10}$	$1 + x + x^3$	L
e_O	$1 + 3x + 4x^2 + 6x^3 + 5x^4 + 2x^5 + x^6 - 7x^8 - 5x^9 - 7x^{10}$	$x + x^2 + x^3$	O

Key Observations from the Example

1. RSA encryption is based on exponentiation and modular arithmetic, which makes it computationally expensive.
2. NTRU relies on polynomial multiplication and modular reduction, making it much faster and efficient.
3. RSA requires a large key size to maintain security, whereas NTRU achieves the same level of security with smaller keys.
4. Decryption in NTRU is more efficient due to lattice-based computations, whereas RSA requires significant processing power.

```

1 import numpy as np
2 N = 11
3 p = 3
4 q = 61
5 def poly_mod(poly, mod):
6     return [coeff % mod for coeff in poly]
7 def poly_add(a, b):
8     return [(ai + bi) for ai, bi in zip(a, b)]
9 def poly_mul(a, b):
10    result = [0] * N
11    for i in range(N):
12        for j in range(N):
13            result[(i + j) % N] += a[i] * b[j]
14    return result
15 def poly_from_expr(expr):
16    coeffs = [0] * N
17    expr = expr.replace('-', '+-')
18    terms = expr.split('+')
19    for term in terms:
20        term = term.strip()
21        if term == '':
22            continue
23        if 'x' not in term:
24            coeffs[0] += int(term)
25        elif term == 'x':
26            coeffs[1] += 1
27        elif term == '-x':
28            coeffs[1] -= 1
29        elif '^' in term:
30            parts = term.split('x^')

```

```

32            power = int(parts[1])
33            if coeff in ('', '+'):
34                coeff = 1
35            elif coeff == '-':
36                coeff = -1
37            else:
38                coeff = int(coeff)
39            coeffs[power] += coeff
40        else:
41            # term like "2x"
42            parts = term.split('x')
43            coeff = parts[0]
44            if coeff in ('', '+'):
45                coeff = 1
46            elif coeff == '-':
47                coeff = -1
48            else:
49                coeff = int(coeff)
50            coeffs[1] += coeff
51    return coeffs
52
53 def display_poly(poly, name=""):
54    terms = []
55    for i, coeff in enumerate(poly):
56        if coeff != 0:
57            if i == 0:
58                terms.append(f"{coeff}")
59            else:
60                terms.append(f"{coeff}x^{i}")
61    poly_str = " + ".join(terms) if terms else "0"

```

```

62-     if name:
63-         print(f"{name}(x) = {poly_str}")
64-     else:
65-         print(poly_str)
66- # Polynomials
67- f = poly_from_expr("1+x^2+x^4-x^6-x^8-x^10")
68- f_p = poly_from_expr("x+2x^2+2x^3+2x^4+x^5+x^7+x^9")
69- f_q = poly_from_expr("31+32x+60x^2+24x^3+21x^4+47x^5+53x^6+40x^7+26x^8+49x^9+45x^10")
70- g = poly_from_expr("1+x^2+x^4-x^6-x^8-x^9")
71- h = poly_from_expr("18+6x+21x^2+16x^3+2x^4+21x^5+x^6+17x^7+30x^8+3x^9+25x^10")
72- r = poly_from_expr("1+x+x^2+x^7")
73- m = poly_from_expr("x+x^2+x^3")
74- fqq = poly_mul(f_q,g)
75- ph=[p * c for c in fqq]
76- h = poly_mod(ph,q)
77- display_poly(h, "h=")
78- rh = poly_mul(r, h)
79- prh=[1 * c for c in rh]
80- e = poly_add(prh, m)
81- e_modq = poly_mod(e, q)
82- display_poly(e_modq, "Encrypted e")
83- a = poly_mul(f, e_modq)
84- a_modq = poly_mod(a, q)
85- a_modq = [c if c <= q//2 else c - q for c in a_modq]
86- display_poly(a_modq, "a = f * e mod q")
87- aa = poly_mul(a_modq, f_p)
88- # b = a mod p
89- b_modp = poly_mod(aa, p)
90- b_modp = [c if c <= p//2 else c - p for c in b_modp]
91- display_poly(b_modp, "b = a mod p")

```

H:

$h(x) = 28 + 15x^1 + 35x^2 + 47x^3 + 32x^4 + 37x^5 + 23x^6 + 16x^7 + 14x^8 + 25x^9 + 33x^{10}$
 Encrypted $e(x) = 58 + 53x^1 + 41x^2 + 52x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$
 $a = f * e \text{ mod } q(x) = 3 + 4x^1 + 6x^2 + 5x^3 + 5x^4 + 1x^5 + -1x^7 + -8x^8 + -4x^9 + -8x^{10}$
 $b = a \text{ mod } p(x) = 1 + 1x^1 + 1x^2$

E:

$h(x) = 28 + 15x^1 + 35x^2 + 47x^3 + 32x^4 + 37x^5 + 23x^6 + 16x^7 + 14x^8 + 25x^9 + 33x^{10}$
 Encrypted $e(x) = 57 + 52x^1 + 41x^2 + 52x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$
 $a = f * e \text{ mod } q(x) = 3 + 2x^1 + 4x^2 + 4x^3 + 4x^4 + 1x^6 + -7x^8 + -3x^9 + -7x^{10}$
 $b = a \text{ mod } p(x) = 1x^2$

L:

$h(x) = 28 + 15x^1 + 35x^2 + 47x^3 + 32x^4 + 37x^5 + 23x^6 + 16x^7 + 14x^8 + 25x^9 + 33x^{10}$
 Encrypted $e(x) = 58 + 53x^1 + 40x^2 + 53x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$
 $a = f * e \text{ mod } q(x) = 2 + 5x^1 + 4x^2 + 5x^3 + 5x^4 + 2x^5 + -1x^6 + -7x^8 + -5x^9 + -7x^{10}$
 $b = a \text{ mod } p(x) = 1 + 1x^1 + 1x^3$

O:

$h(x) = 28 + 15x^1 + 35x^2 + 47x^3 + 32x^4 + 37x^5 + 23x^6 + 16x^7 + 14x^8 + 25x^9 + 33x^{10}$
 Encrypted $e(x) = 57 + 53x^1 + 41x^2 + 53x^3 + 6x^4 + 19x^5 + 3x^6 + 43x^7 + 7x^8 + 29x^9 + 58x^{10}$
 $a = f * e \text{ mod } q(x) = 1 + 3x^1 + 4x^2 + 6x^3 + 5x^4 + 2x^5 + 1x^6 + -7x^8 + -5x^9 + -7x^{10}$
 $b = a \text{ mod } p(x) = 1x^1 + 1x^2 + 1x^3$

References

- [1] Dummett, M. An introduction to the philosophy of mathematics. Oxford University Press, 2004.
- [2] Gallian, J. A. Contemporary abstract algebra (8th ed.). Cengage Learning, 2010.
- [3] Herstein, I. N. Topics in algebra (2nd ed.). John Wiley & Sons, 2006.
- [4] Shoup, V. A computational introduction to number theory and algebra. Cambridge University Press, 2001.
- [5] Lidl, R., & Niederreiter, H. Finite fields (2nd ed.). Cambridge University Press, 1997.
- [6] Hoffstein, Jeffrey, Pipher, Jill, Silverman, Joseph H. NTRU: A Ring-Based Public Key Cryptosystem. Brown University, 1998.
- [7] Clark, Benjamin. Understanding the NTRU Cryptosystem. University of Akron, 2019.
- [8] Koblitz, Neal. Mathematics of Computation, American Mathematical Society, 1987.
- [9] Nitaj, Abderrahmane. The Mathematics of the NTRU Public Key Cryptosystem. Université de Caen Basse Normandie, 2003.
- [10] Hankerson, Darrel; Menezes, Alfred; Vanstone, Scott. Guide to Elliptic Curve Cryptography, Springer, 2004.